SMART CLASSROOM TIMETABLE ALLOCATION

Python Project Report

Problem Statement

Develop a Python program to allocate courses to classrooms and periods. The program represents course details, room capacities and periods using lists and tuples, validates classroom capacity, detects timetable clashes, and checks whether all courses can be accommodated under the given constraints.

Project Overview

The program checks each proposed timetable entry and reports capacity problems, room clashes, repeated course scheduling, and courses that have not been assigned. A timetable is considered fully accommodated when every course is assigned and no constraints are violated.

Submitted as a Python programming project

1. Objectives

• Represent courses and classroom capacities as tuples stored in lists.
• Represent available time periods using a list.
• Assign each course to a room and a period.
• Validate that each room can hold the students enrolled in a course.
• Detect room clashes and duplicate course-period assignments.
• Identify courses that are not scheduled.
• Decide whether all courses are accommodated under the stated constraints.

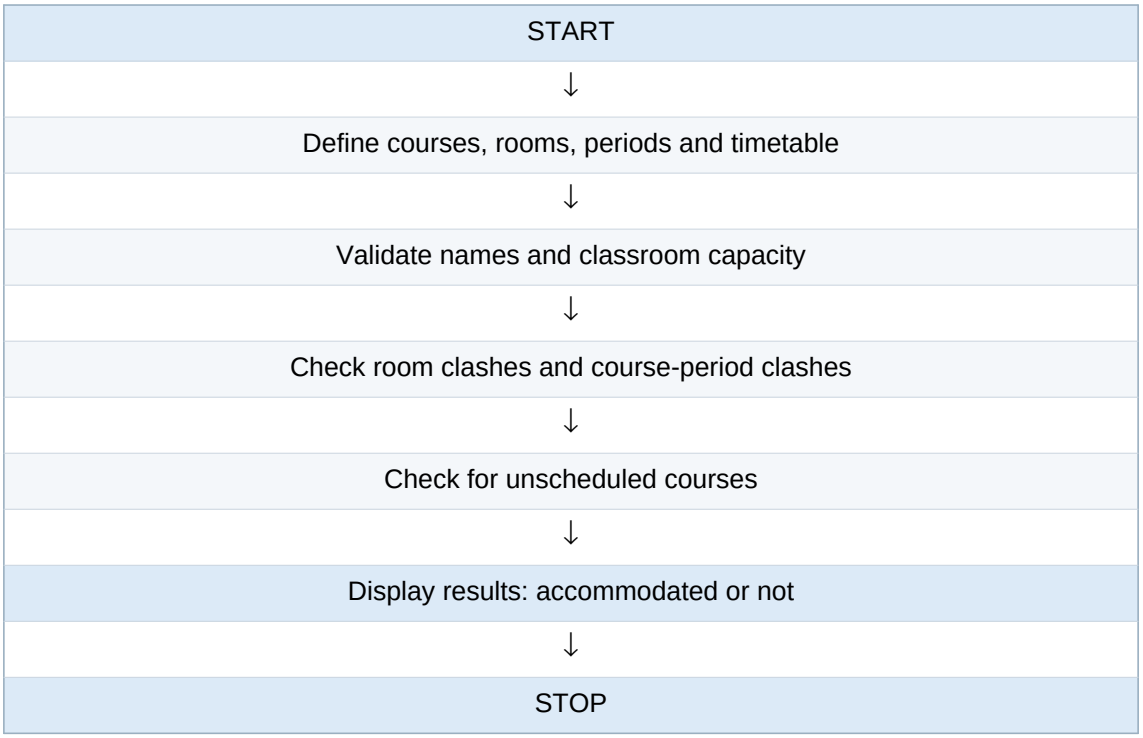
2. Assumptions

• Each course has a known student count.
• Each room has a fixed seating capacity.
• Each timetable entry contains a course, room and period.
• A room can host only one course in a period.
• A course can be scheduled only once in the same period.
• All course, room and period names must match the predefined lists.
• The sample timetable is checked against these constraints; it is not an automatic optimizer that searches every possible allocation.

3. Algorithm

1. Start the program.
2. Define courses, room capacities, periods and proposed timetable entries.
3. Create lookup dictionaries for course sizes and room capacities.
4. Check that every entry uses a known course, room and period.
5. Compare course size with room capacity.
6. Track room-period pairs to detect room clashes.
7. Track course-period pairs to detect repeated scheduling.
8. Find courses missing from the timetable.
9. Display all validation results and stop.

4. Flowchart



5. Pseudocode

```
BEGIN
  DEFINE courses, rooms, periods, timetable
  issues = empty list
  assigned_courses = empty set
  room_periods = empty set
  course_periods = empty set

  FOR each (course, room, period) in timetable:
      validate course, room and period names
      IF course size > room capacity:
          report capacity issue
      IF (room, period) already exists:
          report room clash
      IF (course, period) already exists:
          report course clash
      record valid assignment

  FIND courses not present in assigned_courses
  IF issues is empty AND no courses are missing:
      report all courses accommodated
  ELSE:
      report constraints that failed
END
```

6. Python Source Code

Save this code as **smart_classroom_timetable.py** and run it in VS Code.

```
# Smart Classroom Timetable Allocation
# Courses, rooms and periods are represented using tuples and lists.

courses = [
    ("Python", 40),
    ("DBMS", 35),
    ("Data Structures", 45),
    ("Communication Skills", 30),
]

rooms = [
    ("Room A", 40),
    ("Room B", 50),
    ("Room C", 30),
]

periods = ["09:00-10:00", "10:00-11:00", "11:00-12:00", "12:00-01:00"]

# Each timetable entry is (course, room, period).
timetable = [
    ("Python", "Room A", "09:00-10:00"),
    ("DBMS", "Room B", "09:00-10:00"),
    ("Data Structures", "Room B", "10:00-11:00"),
    ("Communication Skills", "Room C", "10:00-11:00"),
]

def validate_timetable(courses, rooms, periods, timetable):
    course_sizes = dict(courses)
    room_sizes = dict(rooms)
    issues = []
    assigned_courses = set()
    room_periods = set()
    course_periods = set()

    for course, room, period in timetable:
        if course not in course_sizes:
            issues.append(f"Unknown course: {course}")
            continue
        if room not in room_sizes:
            issues.append(f"Unknown room: {room}")
            continue
        if period not in periods:
            issues.append(f"Invalid period: {period}")
            continue

        if course_sizes[course] > room_sizes[room]:
            issues.append(
                f"Capacity issue: {course} needs {course_sizes[course]}, "
                f"but {room} holds {room_sizes[room]}"
            )

        if (room, period) in room_periods:
            issues.append(f"Room clash: {room} is booked more than once at {period}")
        room_periods.add((room, period))

        if (course, period) in course_periods:
            issues.append(f"Course clash: {course} is scheduled more than once at {period}")
        course_periods.add((course, period))
        assigned_courses.add(course)

    missing = [course for course, _ in courses if course not in assigned_courses]
    if missing:
        issues.append("Unscheduled courses: " + ", ".join(missing))

    return issues, missing

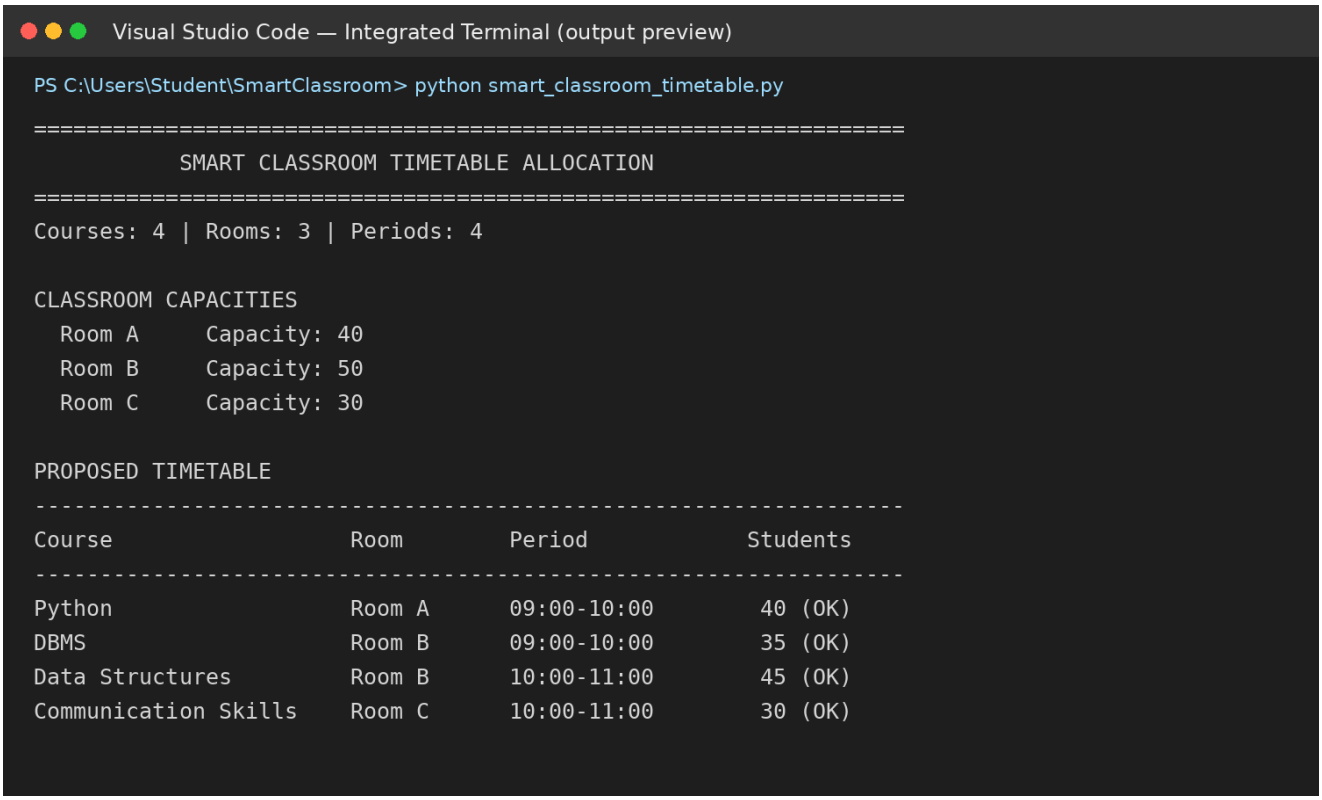
def main():
    print("=" * 66)
    print("           SMART CLASSROOM TIMETABLE ALLOCATION")
    print("=" * 66)
    print(f"Courses: {len(courses)} | Rooms: {len(rooms)} | Periods: {len(periods)}")
    print("\\nCLASSROOM CAPACITIES")
    for room, capacity in rooms:
        print(f"  {room:<10} Capacity: {capacity}")
    print("\\nPROPOSED TIMETABLE")
    print("-" * 66)
    print(f"{'Course':<24}{'Room':<12}{'Period':<18}{'Students'}")
    print("-" * 66)
    room_capacity = dict(rooms)
    course_size = dict(courses)
```

```
    for course, room, period in timetable:
        status = "OK" if course_size[course] <= room_capacity[room] else "OVER"
        print(f"{course:<24}{room:<12}{period:<18}{course_size[course]:>3} ({status})")
    issues, missing = validate_timetable(courses, rooms, periods, timetable)
    print("\\nVALIDATION RESULTS")
    if issues:
        for issue in issues:
            print("  [!] " + issue)
    else:
        print("  [OK] No room clashes detected.")
        print("  [OK] All assigned courses fit classroom capacities.")
        print("  [OK] No course is scheduled twice in the same period.")
    print(f"  Courses accommodated: {len(set(x[0] for x in timetable))}/{len(courses)}")
    if missing:
        print("  [!] Not all courses are accommodated.")
    else:
        print("  [OK] All courses are accommodated.")
    print("\\nAllocation check completed.")

if __name__ == "__main__":
    main()
```

7. Output Screenshot

The following terminal-style preview shows the output produced by running the Python program. It is formatted to resemble a VS Code integrated terminal, but it is not a direct screenshot captured from the VS Code application.



8. Test Cases

Test case	Expected result
All courses assigned; valid room capacities	All courses accommodated if no clashes exist.
Course has more students than room capacity	Capacity issue is reported.
Two courses use the same room and period	Room clash is reported.
Same course is assigned twice in one period	Course clash is reported.
A course is missing from the timetable	Unscheduled course is reported; not all courses accommodated.

9. Conclusion

This Python program uses lists and tuples to organize courses, rooms and periods. It checks classroom capacity, detects timetable clashes, and identifies unscheduled courses. The final validation messages help determine whether the proposed timetable accommodates all courses under the stated constraints.